

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3**



SCROLLABLE LIST

Oleh:

Randy Febrian

NIM. 2310817110013

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 3

Laporan Praktikum Pemrograman Mobile Modul 3: Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Randy Febrian
NIM : 2310817110013

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code.....	8
B. Output Program	14
C. Pembahasan	17
SOAL 2.....	21
A. Penjelasan	21
TAUTAN GIT	22

DAFTAR GAMBAR

Gambar 1. Contah UI List	7
Gambar 2. Contoh UI Detail	7
Gambar 3. Screenshot Home Screen Soal 1	14
Gambar 4. Screenshot Detail Screen Soal 1	15
Gambar 5. Screenshot Browser Setelah Intent Soal 1	16

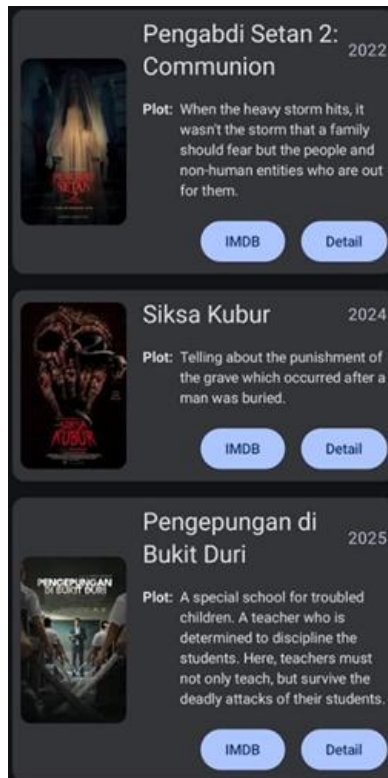
DAFTAR TABEL

Tabel 1. Jawaban Soal 1 MainActivity.kt	8
Tabel 2. Jawaban Soal 1 HomeScreen.kt	8
Tabel 3. Jawaban Soal 1 DetailScreen.kt	10
Tabel 4. Jawaban Soal 1 ItemData.kt	11
Tabel 5. Jawaban Soal 1 Navigation.kt	12
Tabel 6. Jawaban Soal 1 ItemViewModel.kt.....	12

SOAL 1

Soal Praktikum

1. Buatlah sebuah aplikasi Android menggunakan XML atau Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:
 1. List menggunakan fungsi RecyclerView (XML) atau LazyColumn (Compose)
 2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
 3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
 4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component/intent untuk membuka laman detail item
 5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
 6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
 7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
 8. Aplikasi berbasis XML harus menggunakan ViewBinding
2. Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat? UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:



Gambar 1. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 2. Contoh UI Detail

A. Source Code

Tabel 1. Jawaban Soal 1 MainActivity.kt

1	package com.example.myapplication
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	
8	class MainActivity : ComponentActivity() {
9	override fun onCreate(savedInstanceState: Bundle?)
10	{
11	super.onCreate(savedInstanceState)
12	enableEdgeToEdge()
13	setContent {
14	Navigation()
15	}
16	}
17	}

Tabel 2. Jawaban Soal 1 HomeScreen.kt

1	package com.example.scrollablelist
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.Image
6	import androidx.compose.foundation.layout.*
7	import androidx.compose.foundation.lazy.LazyColumn
8	import androidx.compose.foundation.lazy.items
9	import
10	androidx.compose.foundation.shape.RoundedCornerShape
11	import androidx.compose.material3.Button
12	import androidx.compose.material3.Card
13	import androidx.compose.material3.MaterialTheme
14	import androidx.compose.material3.Text
15	import androidx.compose.runtime.Composable
16	import androidx.compose.ui.Alignment
17	import androidx.compose.ui.Modifier
18	import androidx.compose.ui.draw.clip
19	import androidx.compose.ui.platform.LocalContext
20	import androidx.compose.ui.res.painterResource
21	import androidx.compose.ui.layout.ContentScale
22	import androidx.compose.ui.res.stringResource
23	import androidx.compose.ui.unit.dp
24	import androidx.navigation.NavController
25	


```

26 @Composable
27 fun HomeScreen(navController: NavController, viewModel:
28 ItemViewModel) {
29     val context = LocalContext.current
30     Column(modifier = Modifier
31         .fillMaxSize()
32         .padding(16.dp)) {
33         LazyColumn {
34             items(itemList) { item ->
35                 Card(
36                     modifier = Modifier
37                         .padding(8.dp)
38                         .fillMaxWidth()
39                 ) {
40                     Row(modifier =
41 Modifier.padding(16.dp)) {
42                         Image(
43                             painter =
44 painterResource(id = item.pictureId),
45                             contentDescription = null,
46                             contentScale =
47 ContentScale.Crop,
48                             modifier = Modifier
49                                 .height(130.dp)
50                                 .width(100.dp)
51
52 .align(Alignment.CenterVertically)
53
54 .clip(RoundedCornerShape(10.dp))
55                             )
56                         Column(modifier = Modifier
57                             .padding(start = 16.dp)
58                             .fillMaxWidth()) {
59
60                             Text(text = item.name,
61 style = MaterialTheme.typography.titleMedium)
62                             Text(text =
63 stringResource(id = item.description), style =
64 MaterialTheme.typography.bodyMedium)
65
66                             Row(
67                                 modifier = Modifier
68                                     .padding(top =
69 8.dp)
70                                     .fillMaxWidth(),
71                                 horizontalArrangement =
72 Arrangement.SpaceBetween

```

```

73         ) {
74             Button(onClick = {
75
76                 viewModel.setSelectedItem(item)
77
78                 navController.navigate("detail_Screen")
79             }) {
80                 Text("Detail")
81             }
82             Button(onClick = {
83                 val intent =
84                 Intent(Intent.ACTION_VIEW, Uri.parse(item.url))
85
86                 context.startActivity(intent)
87             }) {
88                 Text("Buka Situs")
89             }
90         }
91     }
92 }
93 }
94 }
95 }
96 }
97 }

```

Tabel 3. Jawaban Soal 1 DetailScreen.kt

```

1 package com.example.scrollablelist
2
3 import androidx.compose.foundation.Image
4 import androidx.compose.foundation.layout.Column
5 import androidx.compose.foundation.layout.fillMaxSize
6 import androidx.compose.foundation.layout.fillMaxWidth
7 import androidx.compose.foundation.layout.padding
8 import androidx.compose.material3.MaterialTheme
9 import androidx.compose.material3.Text
10 import androidx.compose.runtime.Composable
11 import androidx.compose.ui.Alignment
12 import androidx.compose.ui.Modifier
13 import androidx.compose.ui.layout.ContentScale
14 import androidx.compose.ui.res.painterResource
15 import androidx.compose.ui.res.stringResource
16 import androidx.compose.ui.unit.dp
17 import androidx.navigation.NavController
18
19 @Composable
20 fun DetailScreen(itemData: ItemData, navController:
21 NavController) {

```

22	Column (Modifier
23	.fillMaxSize()
24	.padding(16.dp)) {
25	Image(painter = painterResource(id =
26	itemData.pictureId),
27	contentDescription = null,
28	contentScale = ContentScale.Crop
29	,modifier = Modifier
30	.fillMaxWidth()
31	.padding(top = 10.dp))
32	Text(text = itemData.name, style =
33	MaterialTheme.typography.titleLarge, modifier =
34	Modifier
35	.padding(vertical = 10.dp)
36	.align(Alignment.CenterHorizontally))
37	Text(text = stringResource(id =
38	itemData.description), style =
39	MaterialTheme.typography.bodyMedium)
40	}
41	}

Tabel 4. Jawaban Soal 1 ItemData.kt

1	package com.example.scrollablelist
2	
3	import android.os.Parcelable
4	import kotlinx.parcelize.Parcelize
5	
6	@Parcelize
7	data class ItemData(
8	val name: String,
9	val pictureId: Int,
10	val url: String,
11	val description: Int
12	) : Parcelable
13	
14	val itemList = listOf(
15	ItemData("Cirrostratus", R.drawable.cirrostratus,
16	"https://id.wikipedia.org/wiki/Awan_sirostratus",
17	R.string.cirrostratus_desc),
18	ItemData("Cirrocumulus", R.drawable.cirrocumulus,
19	"https://id.wikipedia.org/wiki/Awan_sirokumulus",
20	R.string.cirrocumulus_desc),
21	ItemData("Cumulus", R.drawable.cumulus,
22	"https://id.wikipedia.org/wiki/Awan_kumulus",
23	R.string.cumulus_desc),
24	ItemData("Altostratus", R.drawable.altostratus,
25	"https://id.wikipedia.org/wiki/Awan_altostratus",
26	R.string.altostratus_desc),

```

27     ItemData("Cumulonimbus", R.drawable.cumulonimbus,
28 "https://id.wikipedia.org/wiki/Awan_kumulonimbus",
29 R.string.cumulonimbus_desc),
30 )

```

Tabel 5. Jawaban Soal 1 Navigation.kt

```

1 package com.example.scrollablelist
2
3 import androidx.compose.runtime.Composable
4 import androidx.compose.runtime.collectAsState
5 import androidx.navigation.compose.NavHost
6 import androidx.navigation.compose.composable
7 import
8 androidx.navigation.compose.rememberNavController
9
10 @Composable
11 fun Navigation() {
12     val navController = rememberNavController()
13     val viewModel = ItemViewModel()
14
15     NavHost(navController = navController,
16 startDestination = "home_Screen") {
17         composable("home_Screen") {
18             HomeScreen(navController, viewModel)
19         }
20         composable("detail_Screen") {
21             val item =
22 viewModel.selectedItem.collectAsState().value
23             item?.let {
24                 DetailScreen(itemData = it,
25 navController = navController)
26             }
27         }
28     }
29 }

```

Tabel 6. Jawaban Soal 1 ItemViewModel.kt

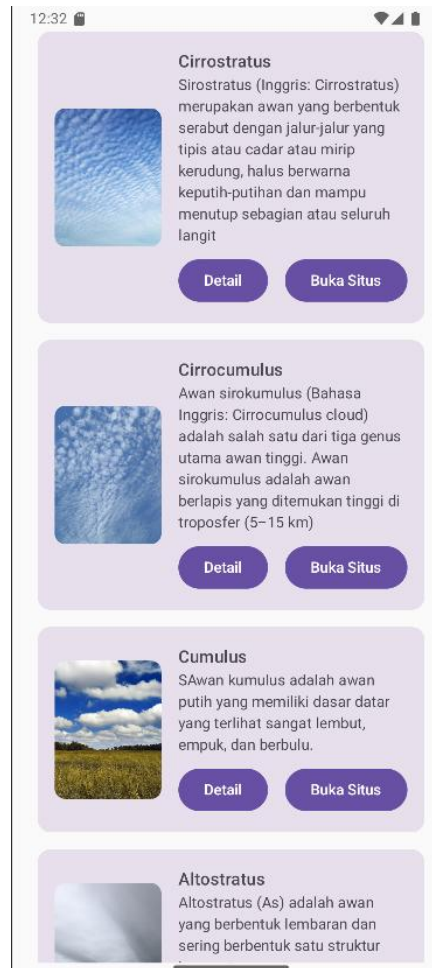
```

1 package com.example.scrollablelist
2
3 import androidx.lifecycle.ViewModel
4 import kotlinx.coroutines.flow.MutableStateFlow
5 import kotlinx.coroutines.flow.asStateFlow
6
7 class ItemViewModel : ViewModel() {
8     private val _selectedItem =
9 MutableStateFlow<ItemData?>(null)
10     val selectedItem = _selectedItem.asStateFlow()
11 }

```

12	fun setSelectedItem(item: ItemData) {
13	_selectedItem.value = item
14	}
15	}

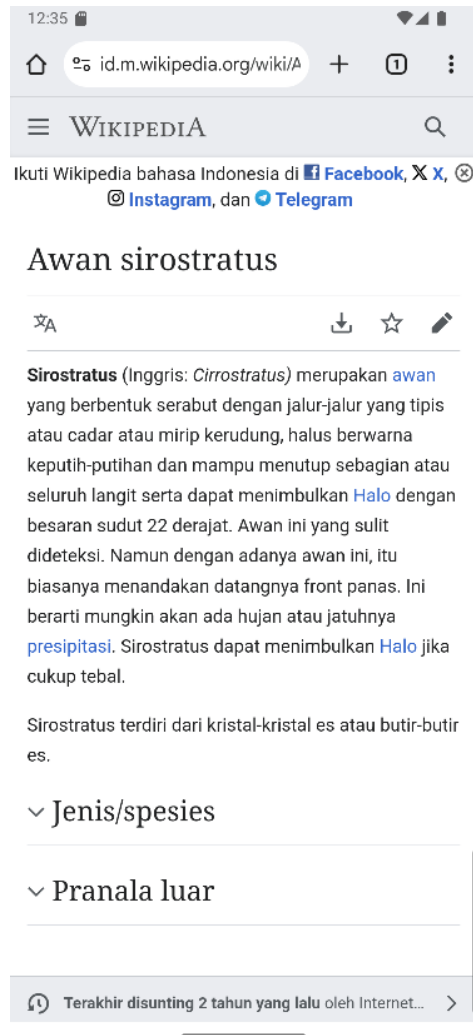
B. Output Program



Gambar 3. Screenshot Home Screen Soal 1



Gambar 4. Screenshot Detail Screen Soal 1



Gambar 5. Screenshot Browser Setelah Intent Soal 1

C. Pembahasan

HomeScreen.kt

Pada baris [1] dengan deklarasi `package com.example.scrollablelist` yang menunjukkan bahwa file ini merupakan bagian dari struktur aplikasi Android tersebut. Selanjutnya, pada baris [3 – 24] berfungsi untuk mengimpor beberapa library untuk UI menggunakan Jetpack Compose, seperti `Intent` dan `Uri` digunakan untuk membuka URL eksternal, sedangkan komponen seperti `Column`, `Row`, `LazyColumn`, `Image`, `Card`, dan `Button` dari Jetpack Compose digunakan untuk menyusun dan menampilkan antarmuka pengguna. Library `painterResource` dan `stringResource` juga diimpor agar gambar dan teks dari folder `res` dapat digunakan dalam komposisi UI.

Pada baris [27 - 28], berfungsi untuk menampilkan halaman utama dari aplikasi. Fungsi ini menerima dua parameter, yaitu `NavController` untuk mengatur navigasi antar layar, dan `viewModel` untuk menyimpan serta mengelola data item. Pada baris [29] berfungsi untuk mengambil konteks aplikasi menggunakan `LocalContext.current`, yang nantinya digunakan untuk menjalankan aktivitas seperti membuka browser eksternal.

Selanjutnya, pada baris [30 - 32] berfungsi untuk membuat sebuah `Column` sebagai layout utama yang mengisi seluruh ukuran layar dan diberi padding sebesar 16dp. Kemudian, baris [33] berfungsi untuk membuat `LazyColumn` yang digunakan untuk menampilkan daftar item secara efisien dan dapat discroll. Kemudian, baris [34 - 39], membuat `items(itemList)` setiap item dari daftar `itemList` akan ditampilkan satu per satu dengan komponen `Card`.

Baris [40 - 41] berfungsi untuk membuat `Row` yang digunakan untuk menyusun elemen secara horizontal, yaitu gambar di sebelah kiri dan informasi item di sebelah kanan. Pada baris [42 – 55] berfungsi untuk menampilkan gambar menggunakan komponen `Image`, dengan ukuran tinggi 130dp dan lebar 100dp serta diberi rounded corner menggunakan `clip(RoundedCornerShape(10.dp))`.

Pada baris [56 – 73] berfungsi untuk menampilkan informasi nama dan deskripsi item ditampilkan menggunakan `Text`. Di bawah informasi tersebut, terdapat dua tombol aksi yang diletakkan dalam satu `Row` dengan pengaturan `horizontalArrangement = Arrangement.SpaceBetween` agar kedua tombol berada di kiri dan kanan secara merata. Kemudian, baris [74 – 81] berfungsi untuk membuat Tombol "Detail" yang menyimpan item yang sedang dipilih ke dalam `viewModel` dan melakukan navigasi ke layar detail dengan

`navController.navigate("detail_Screen")`. Pada baris [82 – 89] berfungsi untuk membuat tombol untuk membuka situs. Tombol "Buka Situs" () akan membuka browser menggunakan Intent untuk mengakses URL yang tertera pada data item (`item.url`).

DetailScreen.kt

Baris [1] berfungsi sebagai penanda bahwa `DetailScreen.kt` merupakan bagian dari package `com.example.scrollablelist`. Baris [3 – 17] berfungsi untuk mengimpor berbagai library dari Jetpack Compose yang dibutuhkan untuk membangun antarmuka pengguna. Library tersebut digunakan untuk menampilkan gambar dan teks, mengatur layout vertikal, serta mengambil sumber daya dari folder `res`, seperti gambar dan string.

Baris [20 – 21] berfungsi untuk mendeklarasikan fungsi `DetailScreen` yang digunakan sebagai tampilan halaman detail dari item yang dipilih. Fungsi ini menerima dua parameter, yaitu `itemData` yang berisi data dari item yang akan ditampilkan, dan `navController` yang digunakan untuk navigasi antar halaman.

Baris [22 – 31] berfungsi untuk membuat layout vertikal menggunakan `Column`. Di dalamnya, gambar dari item ditampilkan di bagian atas layar dengan ukuran yang disesuaikan dengan lebar layar. Gambar tersebut ditampilkan dengan tampilan yang sudut yang melengkung.

Baris [32 - 36] berfungsi untuk menampilkan nama item yang diposisikan di tengah layar dengan ukuran teks yang lebih menonjol agar mudah terlihat oleh pengguna. Baris [37 - 40] berfungsi untuk menampilkan deskripsi dari item dalam bentuk teks biasa. Teks ini diambil dari file sumber daya aplikasi dan ditampilkan tepat di bawah nama item.

ItemData.kt

Baris [1 - 10] berfungsi untuk mengatur layout pada aplikasi android. Kemudian baris [12 – 23] berfungsi untuk menampilkan teks `Dice Roller` dengan warna pink pada bagian atas layer aplikasi, dengan ukuran teks 30 sp dan menggunakan padding atas, kiri, dan bawah untuk memberikan jarak. Kemudian, baris [25 – 43] berfungsi untuk menampilkan gambar dadu kosong dengan size yang sama dengan constraint atas bawah dari parent sehingga berada di tengah, dengan menggunakan chain agar dadu saling terikat bahkan pada saat mode handphone dibuat landscape. Gambar ini nantinya akan diubah secara acak dari kode `MainActivity.kt`. Baris [32 - 42] adalah tombol atau button yang digunakan untuk roll dadu

dengan height dan widthnya disesuaikan dengan teks yang ada dan diletakkan dibawah gambar dadu.

Navigation.kt

Baris [1] berfungsi sebagai penanda bahwa Navigation.kt merupakan bagian dari package com.example.scrollablelist. Baris [3 – 8] berfungsi untuk mengimpor berbagai komponen dari Jetpack Compose yang digunakan untuk mengelola navigasi antar layar. Library ini memungkinkan pembuatan navigasi deklaratif dengan NavHost, serta pengaturan rute menggunakan composable.

Baris [11 – 13] berfungsi untuk mendefinisikan fungsi Navigation yang mengatur alur navigasi pada aplikasi. Di dalamnya, dibuat objek navController untuk mengelola perpindahan antar layar, serta objek viewModel yang digunakan untuk menyimpan dan mengambil data item yang sedang dipilih.

Baris [15 – 29] berfungsi untuk mendefinisikan rute navigasi menggunakan NavHost, dengan halaman awal ditentukan sebagai home_Screen. Rute pertama akan menampilkan HomeScreen, yang menerima navController dan viewModel sebagai parameter. Rute kedua adalah detail_Screen, yang akan menampilkan DetailScreen. Sebelum itu, data item yang disimpan di viewModel diambil menggunakan collectAsState. Jika nilai item tidak null, maka halaman detail akan ditampilkan dengan membawa data item tersebut sebagai parameter.

ItemViewModel.kt

Baris [1] berfungsi sebagai penanda bahwa ItemViewModel.kt merupakan bagian dari package com.example.scrollablelist. Baris [3 – 5] berfungsi untuk mengimpor class dan fungsi yang dibutuhkan dari Jetpack dan Kotlin Coroutines, yaitu ViewModel untuk manajemen data dan StateFlow untuk mengelola data secara reaktif dan terobservasi.

Baris [7] berfungsi untuk mendefinisikan class ItemViewModel yang merupakan turunan dari ViewModel. Baris [8 - 10] mendeklarasikan variabel _selectedItem sebagai MutableStateFlow dengan nilai awal null, yang digunakan untuk menyimpan item yang dipilih. Kemudian, variabel ini dibungkus menjadi selectedItem sebagai StateFlow agar hanya dapat dibaca dari luar dan tidak bisa diubah secara langsung.

Baris [12 – 14] berfungsi untuk mendefinisikan fungsi `setSelectedItem`, yang akan mengatur nilai item yang dipilih ke dalam `_selectedItem`. Fungsi ini nantinya dipanggil saat pengguna memilih salah satu item pada daftar di halaman utama.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Penjelasan

RecyclerView masih digunakan meskipun kodenya lebih panjang dan bersifat boiler-plate dibandingkan LazyColumn karena banyak proyek Android lama yang belum beralih ke Jetpack Compose. Selain itu, RecyclerView memiliki ekosistem yang matang, fitur lengkap seperti LayoutManager, animasi, dan lain-lain. Meskipun LazyColumn lebih ringkas dan modern, kemudian juga tidak semua tim siap beralih karena Compose masih tergolong baru.

TAUTAN GIT

Berikut adalah tautan untuk source code yang telah dibuat.

https://github.com/rnddfn/Pemrograman_Mobile